

Évaluation heuristique et validation par LLM de conjectures en théorie des graphes

42003524 : Ermanno di Giulio

 - Lien du GitHub

1 Résumé

Ce projet de recherche porte sur la génération automatique de conjectures mathématiques en théorie des graphes, en s’inspirant de la démarche de Fun-Search [1]. L’objectif est de produire des inégalités de la forme $Y \leq f(X)$ reliant des invariants de graphes au sein d’une classe donnée, puis d’évaluer leur robustesse par une méthode heuristique de recherche de contre-exemples. Le score attribué à chaque conjecture est le temps, en secondes, qu’il faut à un algorithme de recuit simulé pour réfuter la conjecture, plafonné à 60 secondes. Un score élevé indique une conjecture difficile à invalider. Les conjectures résistantes sont ensuite soumises au modèle de langage Mistral Large [5] afin d’obtenir un avis argumenté sur leur validité. Sur 566 conjectures traitées, 52 (9,2%) ont résisté à 60 secondes de recherche intensive, et 43 ont été jugées vraies par Mistral.

2 Introduction

La théorie des graphes est un domaine des mathématiques discrètes qui modélise des relations entre objets. Un graphe $G = (V, E)$ est composé d’un ensemble de sommets V et d’arêtes E . Les chercheurs s’intéressent depuis longtemps aux inégalités liant différents paramètres, appelés *invariants*, de graphes appartenant à une même famille structurelle. Par exemple, la conjecture de Nordhaus-Gaddum établit que pour tout graphe G à n sommets, $\chi(G) + \chi(\bar{G}) \leq n + 1$, où χ désigne le nombre chromatique. De tels résultats, simples à énoncer mais non triviaux à prouver, constituent le cœur de la théorie des graphes.

Historiquement, ces conjectures sont formulées manuellement par des mathématiciens après observation empirique. Des outils comme Graffiti ou AutoGraphiX ont tenté d’automatiser ce processus dès les années 1990, mais la puissance de calcul et les techniques d’exploration restaient limitées. Graffiti, développé par Fajtlowicz, fonctionnait par génération aléatoire d’inégalités puis

vérification sur une base de graphes ; AutoGraphiX utilisait quant à lui des méthodes de voisinage variable pour explorer l'espace des graphes extrémaux.

Ce projet s'inscrit dans la continuité de ces travaux en proposant un pipeline entièrement automatisé qui génère des conjectures aléatoires, les évalue par recherche locale, les filtre selon leur résistance, et les soumet à un grand modèle de langage pour validation qualitative. L'approche est inspirée de FunSearch [?], qui utilise des LLMs pour guider la recherche de fonctions mathématiques. Contrairement à FunSearch qui fait jouer au LLM un rôle actif dans la génération, notre système l'utilise en aval comme oracle de validation - une simplification délibérée qui constitue également une limite identifiée de l'approche.

3 Objectifs

- Générer automatiquement des conjectures de la forme “*Pour tout graphe de classe C , $Y \leq f(X)$* ” où C est une classe de graphes, Y et X sont des invariants, et f est une fonction choisie dans un ensemble prédéfini.
- Associer à chaque conjecture un score de résistance basé sur le temps nécessaire pour trouver un contre-exemple par recuit simulé, afin d'identifier les conjectures mathématiquement intéressantes parmi un grand nombre de candidats.
- Vérifier préalablement chaque conjecture sur l'ensemble exhaustif des graphes connexes à au plus 8 sommets disponibles en open data [4], pour éliminer rapidement les conjectures triviales avant de lancer la recherche locale.
- Soumettre les conjectures résistantes au modèle Mistral Large [?] afin d'obtenir un avis argumenté (raisonnement structuré étape par étape et verdict VRAI/FAUX) permettant de croiser la mesure heuristique avec un jugement qualitatif.
- Construire une architecture producteur/consommateur permettant de découpler la génération et l'évaluation des conjectures de leur soumission à l'API Mistral, afin de ne pas bloquer le pipeline de génération sur les limites de débit de l'API.

4 Choix des Technologies

4.1 Langage Python et NetworkX

Python a été choisi comme langage principal pour sa richesse en bibliothèques scientifiques et sa lisibilité. NetworkX [2] est la bibliothèque de référence pour la manipulation de graphes en Python : elle fournit nativement le calcul du diamètre, du rayon, des couplages maximaux, des cliques, de la planarité, etc.

Elle supporte également le format Graph6 utilisé par la base de données de Brendan McKay (ANU) [4], ce qui permet de charger directement les quelques milliers (12346) de graphes connexes à au plus 8 sommets avec une seule ligne de code :

```
G = nx.from_graph6_bytes(g6_str.encode())
```

4.2 Mistral Large

L'API Mistral Large [5] a été choisie pour l'étape de validation qualitative. Il s'agit d'un modèle avec de solides capacités de raisonnement mathématique et symbolique. Il est interrogé avec une température de 0.3 pour favoriser des réponses déterministes et reproductibles. Le modèle est invité à répondre en JSON structuré afin de permettre un parsing automatique fiable de son verdict :

```
{
  "raisonnement": "raisonnement etape par etape...",
  "verdict": "VRAI ou FAUX"
}
```

Ce format impose au modèle de raisonner avant de conclure, ce qui améliore la qualité des verdicts par rapport à une simple demande de réponse en un mot.

4.3 Architecture fichiers JSON et sécurité des threads

La communication entre le producteur et le consommateur repose sur deux fichiers JSON : `a.traiter.json` (file d'attente) et `traitees.json` (archive des conjectures traitées). Les accès concurrents sont protégés par un verrou `threading.Lock()` pour la sécurité inter-threads. Les écritures utilisent un fichier temporaire renommé atomiquement via `os.replace()` pour éviter toute corruption en cas d'interruption. Cette technique garantit que le fichier est toujours dans un état cohérent : soit l'ancienne version, soit la nouvelle, jamais à moitié écrite.

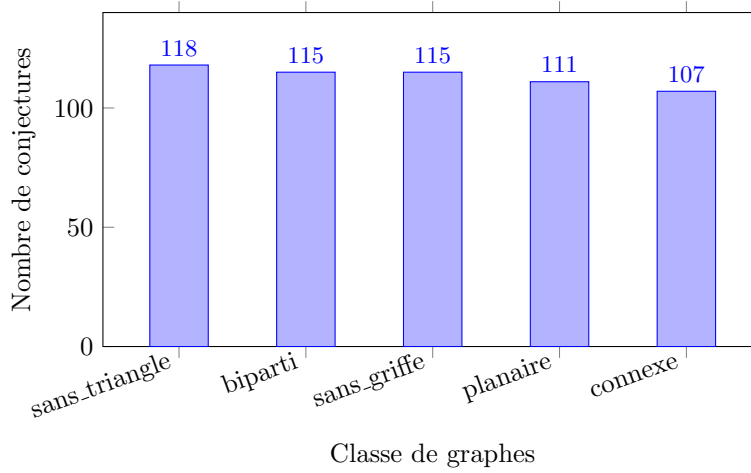
4.4 Bibliothèques Utilisées

- `networkx` [2] : Manipulation de graphes, calcul de tous les invariants.
- `mistralai` [5] : Intégration avec l'API Mistral Large.
- `threading`, `tempfile` : Gestion de la concurrence et écriture atomique.
- `json`, `os` : Persistance des données et gestion des fichiers.
- `time`, `math`, `random`, `copy` : Recuit simulé et mutations de graphes.
- `itertools` : Énumération combinatoire pour le calcul exact de $\alpha(G)$.

5 Méthodologie

5.1 Données traitées

Les conjectures sont générées selon un squelette fixe impliquant des classes de graphes et des invariants. Les graphes de référence pour la validation initiale proviennent de la base de données de Brendan McKay [?], qui recense tous les graphes connexes non isomorphes jusqu'à 8 sommets — soit plusieurs milliers de graphes. La répartition des 566 conjectures traitées par classe est la suivante :



La répartition est quasi-uniforme entre les cinq classes, ce qui est attendu puisque la génération est aléatoire. Les 14 invariants utilisés sont résumés dans le tableau suivant :

Notation	Description
n, m	Nombre de sommets, nombre d'arêtes
$\delta, \Delta, \bar{d}$	Degrés minimum, maximum, moyen
diam, rad	Diamètre, rayon
ρ	Densité ($= 2m/n(n-1)$)
t	Nombre de triangles
ω	Cardinalité de la clique maximum
α	Cardinalité de l'ensemble indépendant maximum
ν	Cardinalité du couplage maximum
γ	Cardinalité de l'ensemble dominant minimum
τ	Cardinalité de la couverture minimum

Table 1: Invariants utilisés dans la génération de conjectures

15 fonctions f sont disponibles, allant des fonctions linéaires simples ($X/4$, $X/3$, $X/2$, $2X/3$, $3X/4$, $X-1$, $X-2$, $X+1$) aux fonctions non-linéaires ($\sqrt{X}$, $\sqrt{X}/2$, $\log_2(X+1)$, $X^{2/3}$, $\lfloor X/2 \rfloor + 1$, $\lceil X/2 \rceil$, $\lfloor \sqrt{X} \rfloor + 1$).

5.2 Les Étapes

5.2.1 Génération des conjectures

Chaque conjecture est générée dans `generate_conjectures.py` en tirant aléatoirement une classe C , deux invariants distincts Y et X , et une fonction f . Le texte lisible est construit automatiquement à partir du nom de la fonction, par exemple `'sqrt(X)'` plutôt que `'<lambda>'`, ce qui rend les résultats interprétables. Pour éviter les trivialités les plus grossières, Y et X sont toujours distincts.

```
def generer_conjecture():
    classe = random.choice(CLASSES)
    Y = random.choice(INVARIANTS)
    X = random.choice([inv for inv in INVARIANTS if inv != Y])
    f, f_str = random.choice(FUNCTIONS)
    texte = f"Pour tout graphe {classe}, {Y} <= {f_str}({X})"
    return {'classe': classe, 'Y': Y, 'X': X,
            'f': f, 'f_str': f_str, 'texte': texte}
```

5.2.2 Calcul des invariants

Les invariants sont calculés dans `invariants.py` via la fonction `calculer_invariants(G)`. Un soin particulier est apporté à l'ensemble indépendant maximum α , pour lequel un calcul exact par énumération combinatoire est effectué. La fonction `maximal_independent_set` de NetworkX ne calcule en effet qu'un ensemble maximal au sens de l'inclusion — c'est-à-dire qu'aucun sommet ne peut y être ajouté — ce qui n'est pas la même chose qu'un ensemble de cardinalité maximale. Cette confusion est une erreur classique. La couverture minimum τ est ensuite déduite gratuitement via le théorème de Gallai :

$$\tau(G) + \alpha(G) = n$$

```
def independance_max_exact(G):
    nodes = list(G.nodes())
    n = len(nodes)
    best = 0
    for r in range(n, 0, -1):
        if r <= best:
            break
        for subset in itertools.combinations(nodes, r):
            if all(not G.has_edge(u, v)
                    for u, v in itertools.combinations(subset, 2)):
                best = r
                break
    return best
```

Cette approche par force brute est raisonnable pour les graphes de petite taille (jusqu'à ~ 20 sommets) rencontrés dans notre pipeline, mais deviendrait trop lente pour de grands graphes.

5.2.3 Validation en deux étapes

Étape 1 — Vérification exhaustive sur les petits graphes La conjecture est testée sur tous les graphes connexes à au plus 8 sommets chargés depuis [4] via `chargement_graphes.py`. Pour chaque graphe G appartenant à la classe de la conjecture, on calcule les invariants $Y(G)$ et $X(G)$ et on vérifie si $Y(G) > f(X(G)) + \varepsilon$ (avec $\varepsilon = 10^{-9}$ pour éviter les erreurs d'arrondi). Si c'est le cas, la conjecture est immédiatement réfutée et le score est fixé à 0. Cette étape est implémentée conjointement dans `validate_conjecture.py` et `recherche_locale.py`.

Étape 2 — Recuit simulé Si aucun petit graphe ne viole la conjecture, une recherche locale par recuit simulé est lancée pendant au plus 60 secondes dans `recherche_locale.py`. On part d'un graphe connexe aléatoire de 10 sommets. À chaque itération, une mutation aléatoire parmi quatre types est appliquée : ajout d'une arête, suppression d'une arête (en préservant la connexité), ajout d'un sommet relié aléatoirement à des voisins existants, suppression d'un sommet (en préservant la connexité). La fonction objectif est :

$$\text{obj}(G) = Y(G) - f(X(G))$$

Un contre-exemple correspond à $\text{obj}(G) > 0$. Le critère d'acceptation de type recuit simulé autorise des dégradations avec probabilité $e^{\Delta/T}$, où T décroît selon $T \leftarrow 0,99 \cdot T$ à chaque itération. En cas de stagnation (50 itérations consécutives sans amélioration), un redémarrage aléatoire est effectué et la température est réinitialisée à 1. Le score de la conjecture est le temps écoulé en secondes avant de trouver un contre-exemple, ou $t_{\max} = 60$ s si aucun n'est trouvé.

```
def calculer_score_conjecture(conjecture, temps_max=60):
    # Etape 1 : parcours exhaustif sur les petits graphes
    for G in charger_petits_graphes(8):
        if verifier_classe(G, conjecture['classe']):
            inv = calculer_invariants(G)
            try:
                fX = conjecture['f'](inv[conjecture['X']])
                if inv[conjecture['Y']] > fX + 1e-9:
                    return 0.0 # Contre-exemple immediat
            except Exception:
                pass
    # Etape 2 : recuit simule
    temps_trouve, _ = recherche_contre_exemple(conjecture, temps_max)
    return temps_trouve if temps_trouve else float(temps_max)
```

5.2.4 Vérification de l'appartenance à une classe

L'appartenance à une classe est vérifiée dans `validate_conjecture.py` via la fonction `verifier_classe(G, classe)`. NetworkX fournit nativement les tests de connexité, bipartition, planarité et détection de triangles. La classe `sans_griffe` mérite une attention particulière : un graphe est *sans griffe* (claw-free) si et seulement s'il ne contient pas $K_{1,3}$ comme sous-graphe induit, c'est-à-dire qu'aucun sommet n'a trois voisins deux-à-deux non adjacents. La vérification originale du projet comportait un bug dans la logique de boucle imbriquée : elle pouvait classer incorrectement certains graphes. La version corrigée est :

```
def est_sans_griffe(G):
    for v in G.nodes():
        voisins = list(G.neighbors(v))
        for i in range(len(voisins)):
            for j in range(i+1, len(voisins)):
                if G.has_edge(voisins[i], voisins[j]): continue
            for k in range(j+1, len(voisins)):
                if (not G.has_edge(voisins[i], voisins[k]) and
                    not G.has_edge(voisins[j], voisins[k])):
                    return False # K_{1,3} trouve
    return True
```

Cette correction est vérifiable sur $K_{1,3}$ lui-même (qui doit retourner `False`), C_5 (qui doit retourner `True`), et K_4 (qui doit retourner `True` puisque tout triplet de voisins est relié).

5.2.5 Architecture producteur/consommateur

Le pipeline est décomposé en deux processus indépendants communiquant via des fichiers JSON gérés par `gestion_file.py` :

- **Producteur** (`producteur_conjectures.py`) : génère des conjectures aléatoires, calcule leur score (ce qui prend jusqu'à 60 secondes par conjecture), et n'ajoute à la file `a_traiter.json` que celles dont le score dépasse 5 secondes. Les conjectures trivialement réfutées sont ainsi éliminées avant même d'atteindre Mistral.
- **Consommateur** (`consommateur_mistral.py`) : dépile la file d'attente, interroge l'API Mistral Large avec un prompt demandant une réponse JSON structurée, nettoie et parse la réponse, puis sauvegarde le verdict et le raisonnement dans `traitees.json`. En cas d'échec du parsing, la conjecture est simplement ignorée sans être sauvegardée.

Cette séparation est importante car les deux processus ont des contraintes très différentes : le producteur est limité par le temps de calcul du recuit simulé (60 s par conjecture), tandis que le consommateur est limité par le débit de l'API Mistral (environ 1 requête par seconde).

5.3 Métriques d'évaluation

5.3.1 Score de résistance

La métrique principale est le score de résistance, défini comme le temps en secondes avant qu'un contre-exemple soit trouvé. Plus formellement, pour une conjecture C et un temps limite $t_{\max}$:

$$S(C) = \begin{cases} 0 & \text{si contre-exemple sur petits graphes} \\ t_{\text{trouvé}} & \text{si contre-exemple trouvé par recuit} \\ t_{\max} & \text{sinon (conjecture résistante)} \end{cases}$$

Un score élevé indique une conjecture difficile à réfuter et donc potentiellement intéressante mathématiquement. À noter que ce score mesure la *difficulté heuristique* et non la *vérité* de la conjecture.

5.3.2 Verdict Mistral

Le verdict de Mistral Large constitue une seconde métrique qualitative, indépendante du score heuristique. On calcule le taux de verdicts VRAI parmi les conjectures soumises :

$$P_{\text{vrai}} = \frac{N_{\text{vrai}}}{N_{\text{total}}} \times 100$$

Les conjectures jugées à la fois résistantes (score = 60 s) *et* vraies par Mistral constituent les candidates les plus prometteuses pour une investigation mathématique approfondie.

5.4 Défis et contretemps

5.4.1 Corruption des fichiers JSON

Un défi majeur a été la corruption des fichiers `traitees.json` lors d'interruptions inopinées du programme. Lorsque le processus était interrompu (Ctrl+C ou crash système) exactement pendant une écriture de fichier, le JSON se retrouvait tronqué à mi-chemin, provoquant un `JSONDecodeError` au redémarrage. La solution adoptée est une écriture atomique : on écrit dans un fichier temporaire, puis on le renomme en écrasant l'ancien. Sur Linux, `os.replace()` est atomique, ce qui garantit que le fichier est toujours dans un état cohérent.

5.4.2 Parsing des réponses Mistral

Même avec une orchestration robuste (en utilisant le principe de Chain-of-Thought, Cot [6]), demandant explicitement un JSON sans balises markdown, Mistral Large encapsule parfois sa réponse dans des blocs `““json ... ““`. Cela provoquait des `JSONDecodeError` lors du parsing. La solution a été de nettoyer systématiquement la réponse avant de la parser :

```

raw = response.choices[0].message.content
clean = (raw.strip()
         .removeprefix("'''json")
         .removeprefix("'''")
         .removesuffix("'''")
         .strip())
parsed = json.loads(clean)

```

5.4.3 Distribution bimodale des scores

Une observation surprenante lors de l'exécution a été que la quasi-totalité des scores valaient soit 0 s soit 60 s, sans valeurs intermédiaires. Cela s'explique par la nature discrète des invariants : la fonction objectif $\text{obj}(G) = Y(G) - f(X(G))$ est discontinue, et une seule mutation (ajout ou suppression d'une arête ou d'un sommet) peut faire passer obj de -5 à $+3$ sans gradient continu à suivre. Une conjecture est donc soit trivialement fausse (un contre-exemple existe sur un petit graphe commun et la recherche le trouve quasi-instantanément), soit structurellement résistante (son contre-exemple n'existe peut-être pas, ou nécessite une structure de graphe très spécifique que la recherche aléatoire n'atteint pas dans le délai imparti).

6 Avancement et Résultats

6.1 Distribution des scores

Les résultats de l'évaluation des 566 conjectures sont résumés dans le tableau suivant :

Score	Nombre	Proportion
= 0 s (contre-exemple immédiat)	496	87,6%
$\in]0, 60[$ s (intermédiaire)	18	3,2%
= 60 s (résistante)	52	9,2%
Total	566	100%

Table 2: Distribution des scores de résistance sur 566 conjectures

La distribution est fortement bimodale, comme anticipé. Les 18 conjectures à score intermédiaire représentent les cas les plus intéressants algorithmiquement : leur contre-exemple existe mais nécessite une exploration non triviale.

6.2 Conjectures résistantes

Parmi les 52 conjectures ayant résisté à 60 secondes de recherche, plusieurs impliquent la densité dans des classes structurées. Les plus notables sont présentées ci-dessous :

Conjecture	Score	Mistral
$\forall G$ planaire : $\rho \leq \lfloor \omega/2 \rfloor + 1$	60 s	VRAI
$\forall G$ planaire : $\rho \leq \log_2(\omega + 1)$	60 s	VRAI
$\forall G$ sans_griffe : $\rho \leq \lfloor \sqrt{d} \rfloor + 1$	60 s	FAUX
$\forall G$ biparti : $\rho \leq \log_2(\omega + 1)$	60 s	VRAI
$\forall G$ sans_griffe : $\tau \leq n - 1$	60 s	FAUX
$\forall G$ connexe : $\gamma \leq \log_2(\alpha + 1)$	51,4 s	VRAI
$\forall G$ connexe : $\gamma \leq \lfloor \sqrt{\alpha} \rfloor + 1$	15,8 s	VRAI
$\forall G$ connexe : $\gamma \leq \lceil \alpha/2 \rceil$	5,0 s	VRAI

Table 3: Exemples de conjectures résistantes, leurs scores et verdicts Mistral

Les trois dernières conjectures liant γ (domination minimum) à α (indépendance maximum) dans les graphes connexes sont particulièrement intéressantes : elles forment une hiérarchie cohérente ($\lceil \alpha/2 \rceil \leq \lfloor \sqrt{\alpha} \rfloor + 1 \leq \log_2(\alpha + 1)$ pour α grand), toutes trois jugées vraies par Mistral, ce qui suggère une relation structurelle réelle entre ces deux invariants.

6.3 Verdicts Mistral

Les verdicts de Mistral Large sur les 566 conjectures sont présentés dans le tableau suivant :

Verdict	Nombre	Proportion
FAUX	523	92,4%
VRAI	43	7,6%

Table 4: Verdicts de Mistral Large sur 566 conjectures

Parmi les 43 conjectures jugées VRAI, certaines sont triviales (par exemple $\text{nb_triangles} \leq f(X)$ dans la classe `sans_triangle` où $\text{nb_triangles} = 0$ par définition). D'autres en revanche sont non triviales et résistantes, comme les conjectures reliant ρ à ω dans les graphes planaires ou γ à α dans les graphes connexes.

7 Discussion

Les résultats obtenus mettent en lumière des aspects intéressants du pipeline, tant sur la qualité des conjectures générées que sur les limites fondamentales de l'approche.

7.1 Biais de Mistral vers FAUX

Le taux de réponse FAUX de 92,4% s'explique par deux causes principales. Premièrement, certaines combinaisons (Y, classe) sont triviales par définition

: `nb_triangles` dans `sans_triangle` vaut toujours 0, rendant toute inégalité $0 \leq f(X)$ vraie si $f(X) \geq 0$. Mistral identifie parfois correctement cette trivialité mais conclut quand même FAUX, vraisemblablement parce que la formulation inhabituelle de la conjecture le désoriente. Deuxièmement, pour des conjectures mathématiques inédites formulées aléatoirement, le modèle semble appliquer un biais a priori vers FAUX. Ce biais est statistiquement raisonnable (la grande majorité des inégalités aléatoires sont effectivement fausses) mais il rend les verdicts FAUX peu informatifs.

Pour améliorer la qualité des réponses, il serait utile d'ajouter un filtre de trivialité avant de soumettre les conjectures à Mistral. Les paires (Y, classe) trivialement vraies (comme `nb_triangles` dans `sans_triangle`) pourraient être exclues dès la génération, ce qui éviterait d'envoyer des conjectures inintéressantes à l'API.

7.2 Résistance et intérêt mathématique

Les conjectures liant γ à α dans les graphes connexes (scores 5, 15,8 et 51,4 s) sont les plus prometteuses du lot. La conjecture $\gamma \leq \lceil \alpha/2 \rceil$ est d'ailleurs jugée VRAI par Mistral avec un raisonnement convaincant invoquant des résultats connus sur les graphes connexes. En théorie des graphes, il est connu que $\gamma(G) \leq \alpha(G)$ pour tout graphe connexe, et la borne $\lceil \alpha/2 \rceil$ constituerait une amélioration significative de ce résultat si elle s'avérait vraie.

7.3 Limites de l'approche

Les difficultés rencontrées soulignent plusieurs limites fondamentales. Premièrement, le système ne prouve jamais qu'une conjecture est vraie : un score de 60 secondes signifie uniquement que l'algorithme n'a pas trouvé de contre-exemple dans le délai imparti, pas que la conjecture est valide. Deuxièmement, la génération est purement aléatoire et ne guide pas la recherche vers des conjectures plus prometteuses. Une approche FunSearch authentique utiliserait le LLM pour proposer de nouvelles fonctions f guidées par les résultats précédents, créant une boucle évolutionnaire. Troisièmement, les redémarrages aléatoires en cas de stagnation effacent tout le progrès accumulé : si le contre-exemple à une conjecture n'existe que pour des graphes de grande taille, la recherche ne l'atteindra probablement jamais avec des redémarrages sur des graphes de 10 sommets.

En ce qui concerne les pistes d'amélioration, plusieurs directions sont envisageables : ajouter un filtre de trivialité excluant les paires (Y, classe) redondantes ; implémenter une vraie boucle évolutionnaire où Mistral propose de nouvelles fonctions f basées sur les conjectures résistantes ; augmenter la taille initiale des graphes dans le recuit simulé pour explorer des contre-exemples de plus grande taille ; et étendre la validation exhaustive au-delà de 8 sommets, idéalement jusqu'à 10 ou 12 sommets.

8 Conclusion

Ce projet a permis de concevoir et d'implémenter un pipeline complet de génération et d'évaluation automatique de conjectures en théorie des graphes. L'architecture producteur/consommateur, la validation en deux étapes (petits graphes puis recuit simulé), et l'intégration de Mistral Large constituent une approche originale combinant recherche heuristique et intelligence artificielle.

Sur 566 conjectures traitées, 52 ont résisté à 60 secondes de recherche intensive, dont plusieurs impliquant la densité bornée par une fonction de la clique maximum dans les graphes planaires ou bipartis. Les trois conjectures liant γ à α dans les graphes connexes (scores de 5 à 51 secondes) sont particulièrement prometteuses et méritent une investigation mathématique approfondie. Elles forment une hiérarchie cohérente et sont toutes jugées vraies par Mistral Large, ce qui constitue une convergence encourageante entre la résistance algorithmique et le jugement qualitatif du LLM.

Les enseignements tirés ouvrent la voie à des recherches futures. En particulier, l'implémentation d'une vraie boucle FunSearch où le LLM joue un rôle actif dans la génération des fonctions f - et non uniquement dans la validation - permettrait d'explorer l'espace des conjectures de manière beaucoup plus dirigée et efficace. Ce projet pose ainsi les bases d'une approche automatisée de la découverte mathématique en théorie des graphes.

References

- [1] DeepMind, *FunSearch: Making new discoveries in mathematical sciences using Large Language Models*. Disponible : <https://deepmind.google/blog/funsearch-making-new-discoveries-in-mathematical-sciences-using-large-language-models/>
- [2] A. Hagberg, D. Schult, P. Swart, *Exploring network structure, dynamics, and function using NetworkX*. Disponible : http://conference.scipy.org.s3-website-us-east-1.amazonaws.com/proceedings/scipy2008/paper_2/full_text.pdf
- [3] DeepWiki, *kitft/funsearch*. Disponible : <https://deepwiki.com/kitft/funsearch/1-overview>
- [4] DeepWiki, *kitft/funsearch*. Disponible : <https://users.cecs.anu.edu.au/~bdm/data/graphs.html>
- [5] Mistral AI, *Mistral Large*. Disponible : <https://mistral.ai>
- [6] Google Research - Brain Team, *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Disponible : <https://arxiv.org/pdf/2201.11903>